12 | 实操课：工程搭建全流程演示

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

这是工程搭建篇的实操课。08 到 11 讲，我们完成了 Hify 从零到能跑的全部工程搭建——后端 Maven 骨架、公共基础设施、前端 Vue 工程、UI 设计、基础组件。这节实操课，我会把这整个过程完整演示一遍，让你看到真实的操作节奏：我给了什么指令、Claude Code 怎么回、我怎么判断和追问、哪里顺、哪里卡住了、怎么解决的。

本节实操课将以文字 + 视频的方式进行，主要包含两部分：

第一部分：我在工程搭建过程中真实的体验和看法，以及几个大家比较关心的问题。

第二部分：屏录演示，分五个场景，完整覆盖 08～11 讲的核心内容。

我的真实体验和看法

工程搭建这件事，在 AI Coding 之前我做过很多次。Spring Boot 多模块、Vue 脚手架、Docker 配置、启动脚本，这些东西我闭着眼睛都能搭。但以前搭一套完整的工程至少要半天，如果加上前端、设计系统、基础组件，一两天不算多。

这次用 Claude Code 搭 Hify，我计了一下时间，大概两个多小时，前后端全部就绪，包括 UI 设计。

但我想说的不只是“快了多少”，而是快在哪里、慢在哪里，这比一个数字更有参考价值。

快在哪里：

模板性工作全程不需要思考。Maven 多模块的 pom 结构、MyBatis-Plus 的分页插件配置、logback-spring.xml 的环境区分、线程池的 ThreadPoolExecutor 写法，这些东西我脑子里有，但每次都要查参数、查语法。有了 Claude Code，这类工作几乎不占时间。你描述清楚要什么，它生成，你 review，done。

慢在哪里：

两类情况让我多花了时间。第一类是我描述不够清楚，Claude Code 给了一个“看起来合理但其实不符合我意图”的方案。比如我让它生成线程池配置，没有说明线程名前缀的重要性，它给的 threadFactory 是默认的，调了之后日志里全是 pool-1-thread-3，没法区分。这个问题不在 Claude Code，在我的指令。第二类是启动报错排查，Claude Code 给出的修复方案偶尔是错的，需要来回两三轮才解决。这个是模型本身的局限，接受就行，后面会越来越好。

最让我惊喜的：

UI 设计这块。我是典型的后端工程师，对前端审美没有信心。但按照上一讲的方法，把产品定位、用户群体、风格参考描述清楚，Claude Code 给出的设计系统我很满意。侧边栏的深色 + 渐变主色的科技感，比我自己乱试颜色好多了。这让我意识到，很多以前觉得“需要特定技能才能做的事情”，现在只需要你能描述清楚你想要什么。

下面展开三个问题，是课程里大家比较关心的。

1. 任务拆解到什么粒度才合适?

08 讲提到了任务拆解的两个标准：生成量是否超出一次 review 的范围、步骤间是否有依赖关系。但实际操作时，很多人不确定该拆多细。

我的经验是：以“一次能验证”为单位。

什么叫一次能验证？就是每个子任务做完，你能跑一个命令或者访问一个接口，确认这个子任务是对的。

> Maven 骨架搭完 → `mvn compile` 能通过
>
> hify-common 搭完 → 没有编译错误
>
> 业务空壳搭完 → Spring Boot 能启动
>
> 健康检查接口加完 → `curl localhost:8080/api/v1/health` 返回 200

每个验收点都是一个“锚”，确认到这里是对的，往后出问题也知道问题在锚点之后。

拆得太粗，生成量大、验收困难、问题难定位。拆得太细，频繁切换上下文、效率反而低。所以，以一次能验证为颗粒度，一般是合适的。

2. Claude Code 报错了，怎么处理最高效?

08 讲演示了 `mvn clean install` 报错后，让 Claude Code 修复的过程，有同学问为什么要多轮才解决。

原因在于：Claude Code 修复报错的方式是根据错误信息推断原因，有时候推断不准确，给的修复方案解决了表面问题，但没解决根本原因，导致出现新的报错。

高效处理的方式是：把完整的错误上下文给它，而不只是最后一行报错。

差的方式：

```
报错了，帮我修复
```

好的方式：

```
mvn clean install 报错了，完整错误如下：
[ERROR] ...（完整错误日志）
项目结构是 Maven 多模块，hify-common 被 hify-app 依赖，相关 pom 如下：
...（相关 pom 内容）
帮我分析根本原因并修复。
```

完整上下文 + 相关代码 + 明确问一个“根本原因”，Claude Code 的修复准确率会高很多。如果两三轮还没解决，停下来自己看一眼错误信息，往往问题比你想象的简单——可能只是一个 Bean 注入失败，或者一个依赖版本冲突。

3. 前端 UI 打磨，怎么说才能让 Claude Code 改到位？

11 讲说了“UI 细节打磨才是真功夫”，但很多后端工程师卡在这里，看出来哪里不对，但说不清楚怎么改。

有一个通用句式很管用：

> [位置 / 组件] + [哪里不对] + [应该怎样]

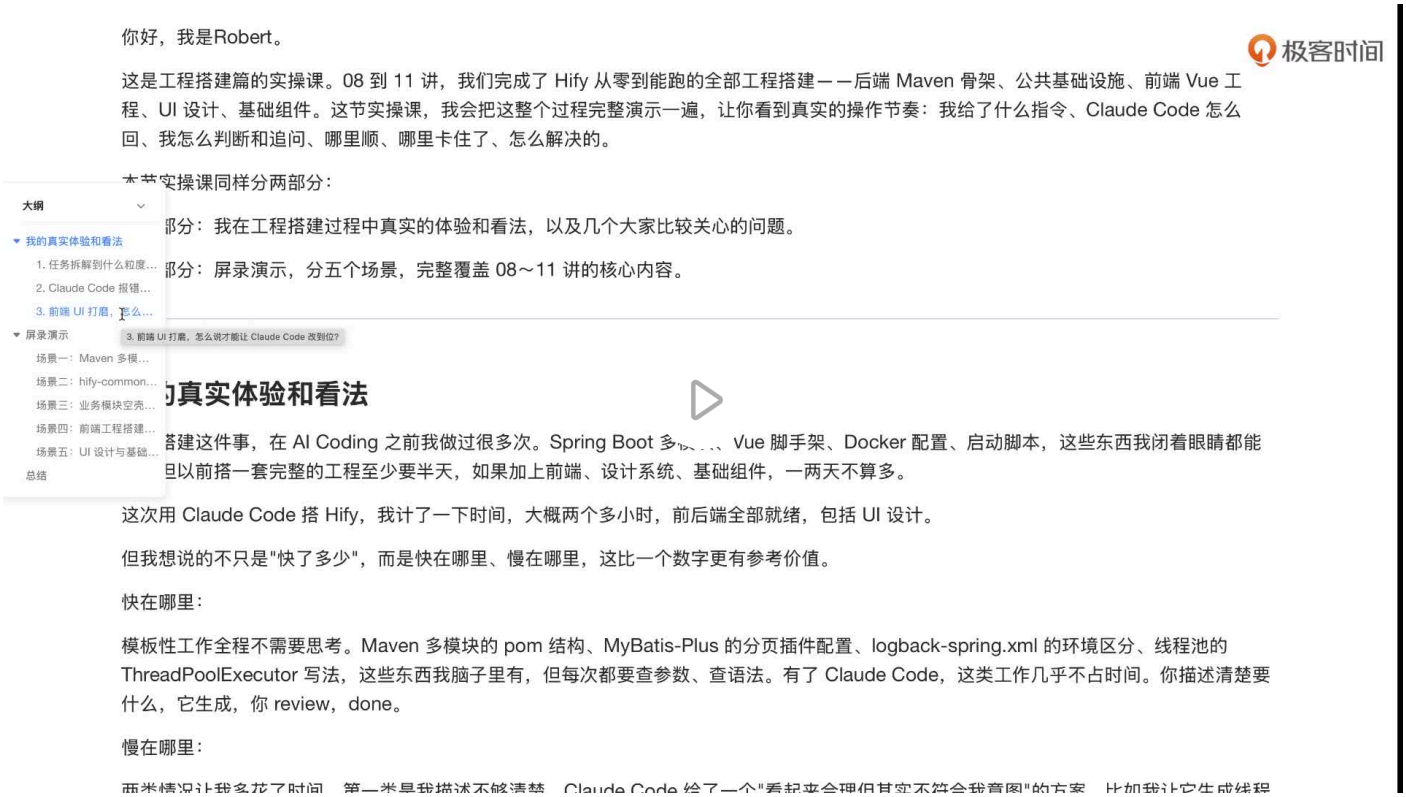
举几个例子：

> 表格和顶部标题区之间间距太大，改成 16px
>
> 操作列的删除按钮颜色用红色，但不要 danger 类型的背景，用 text 类型红色文字
>
> 侧边栏菜单项选中态太不明显，选中的菜单项左侧加一条 3px 的蓝色竖线
>
> 弹窗提交按钮在 loading 时要有转圈动画，不能让用户以为没反应

不需要知道 CSS 属性名，不需要知道 Element Plus 的组件 API，只需要描述“在哪里、什么问题、我想要什么效果”，Claude Code 负责把你的描述翻译成代码。

屏录演示

下面进入屏录部分，分五个场景，完整覆盖工程搭建的核心操作。



场景一：Maven 多模块骨架搭建

目标：从空目录到 Maven 多模块结构就绪，`mvn compile` 能通过。

第一步：确认 CLAUDE.md 已就绪

指令：

```
查看当前 CLAUDE.md，确认项目结构和模块划分部分是否清晰
如果有遗漏或不清晰的地方告诉我。
```

要点：

工程初始化开始前，先让 Claude Code 读一遍 CLAUDE.md，确认它理解了约束

这一步不让它做任何事，只是确认“认知对齐”

Claude Code 的反馈能帮你发现 CLAUDE.md 里的模糊表述

第二步：生成父 pom 和子模块结构

指令：

```
按照 CLAUDE.md 中定义的项目结构，创建 Hify 的 Maven 多模块工程骨架。
父 pom 声明所有子模块，dependencyManagement 统一管理版本。
只创建 pom 文件和目录结构，不需要任何 Java 代码。
```

要点：

“只创建 pom 和目录结构，不需要 Java 代码”是关键约束，不说会生成一堆不需要的代码

拿到输出后检查三件事：模块声明和目录是否对应、依赖关系是否正确、版本管理是否统一

展示 review 过程：对照 CLAUDE.md 里的模块列表，逐一核对 `<modules>` 声明

第三步：验收

```
mvn compile
```

看到 `BUILD SUCCESS`，Maven 骨架完成。有报错直接贴给 Claude Code，带上完整错误信息。

场景二：hify-common 公共基础设施

目标：五个子任务串行完成，每个任务做完编译验证，最后全部就绪。

子任务一：Result 和 PageResult

指令：

```
在 hify-common 中创建统一响应类 Result<T> 和分页响应类 PageResult<T>。
Result 包含 code、message、data，提供 ok() 和 fail() 静态方法。
PageResult 继承 Result，额外包含 total、page、size。
放在 com.hify.common.web 包下。
```

复制代码

要点：

这是最简单的子任务，Claude Code 基本不会出错

展示 review：字段名和 CLAUDE.md 定义的接口规范一致吗?

子任务二：ErrorCode 和 BizException

指令：

```
在 hify-common 中创建错误码枚举 ErrorCode 和业务异常类 BizException。
ErrorCode 包含 PARAM_ERROR、UNAUTHORIZED、NOT_FOUND、INTERNAL_ERROR 等常用错误码。
BizException 持有 ErrorCode，支持自定义 message 覆盖。
放在 com.hify.common.exception 包下。
```

复制代码

子任务三：GlobalExceptionHandler

指令：

```
在 hify-common 中创建全局异常处理器 GlobalExceptionHandler（@RestControllerAdvice）。
捕获 BizException 返回对应 ErrorCode，
捕获 MethodArgumentNotValidException 返回 PARAM_ERROR 和具体校验信息，
兜底捕获 Exception 返回 INTERNAL_ERROR。
所有响应必须使用 Result.fail() + ErrorCode 枚举，禁止硬编码错误码。
```

复制代码

要点：

演示 SDD 闭环：展示 Claude Code 第一版在兜底 catch 里硬编码了 `code: 500, message: "系统繁忙"` 的问题

让它改成 `Result.fail(ErrorCode.INTERNAL_ERROR)`

同步在 CLAUDE.md 补一条：“异常处理必须使用 ErrorCode 枚举，禁止硬编码错误码”

这就是规范迭代的实际过程，不是一次性写完的

子任务四和五：MyBatis-Plus + Redis 配置

指令展示 Claude Code 生成的配置类，重点说明：

MyBatis-Plus 的自动填充处理器

Redis 的 Jackson 序列化配置

两个配置都是高频模板，Claude Code 质量很高，基本不需要改。

场景三：业务模块空壳与启动验证

目标：搭完所有业务模块空壳，Spring Boot 能启动，健康检查返回 200。

第一步：业务模块 package 结构

指令：

```
为 hify-provider、hify-agent、hify-chat、hify-mcp、hify-knowledge、hify-workflow
创建标准的 package 结构。
按 CLAUDE.md 代码组织规范，每个模块包含 web/api/domain/infra 四个包，
每个包里创建一个空的占位类（在类上加注释说明这个包的职责）。
不需要任何业务代码。
```

第二步：Spring Boot 启动类和配置文件

指令：

```
在 hify-app 中创建 Spring Boot 启动类 HifyApplication。
@SpringBootApplication + @MapperScan("com.hify.**.mapper")。
创建 application.yml，配置数据库、Redis、MyBatis-Plus、端口 8080。
数据库连接信息用 localhost 本地开发配置。
```

要点：

展示 `@MapperScan` 扫描路径的重要性，不加这个所有 Mapper 都注入不了

演示 08 讲里的那个报错场景：启动时 `mvn spring-boot:run` 失败，命令用错了

正确命令是 `mvn spring-boot:run -pl hify-app`

第三步：健康检查接口 + 验收

指令：

```
在 hify-app 中创建 HealthController，
路径 GET /api/v1/health，返回 Result.ok("Hify is running")。
```

```
mvn spring-boot:run -pl hify-app
```

```
curl http://localhost:8080/api/v1/health
```

期望输出：

```
{"code": 200, "message": "success", "data": "Hify is running"}
```

场景四：前端工程搭建与前后端联通

目标：Vue 工程搭好，axios 封装完成，调健康检查接口能看到绿色“后端已连接”。

第一步：Vue 工程骨架

指令：

```
初始化 Hify 前端项目 hify-web。
技术栈：Vue 3 + TypeScript + Vite + Element Plus。
Vite 代理：/api 请求转发到 localhost:8080。
目录结构按 CLAUDE.md 中定义的前端结构来。
```

要点：

展示 Vite 代理配置，解释为什么开发阶段不需要 Nginx

目录结构 review：src/api、src/views、src/components、src/utils、src/composables

第二步：axios 统一请求层

指令：

```
在 hify-web/src/utils/ 下创建 request.ts，封装 axios 实例。
baseURL 设为 /api。
响应拦截器：code === 200 直接返回 data 字段（自动解包），
非 200 用 ElMessage.error 提示 message 并 reject。
导出 get、post、put、del 四个方法。
```

要点：

重点说明自动解包的价值：业务代码不需要每次写 `.data`

展示和后端 Result 格式的对应关系

这是前后端规范对齐的体现，一份 CLAUDE.md，两端都在遵守

第三步：路由、页面空壳、前后端联通

指令：

复制代码

```
配置 Vue Router，创建三个路由和对应空壳页面：
模型管理（/providers）、Agent 管理（/agents）、对话（/chat）。
App.vue：左侧 Element Plus 菜单栏，右侧 router-view。
```

然后改造 ProviderList.vue 调健康检查接口，展示联通效果。

场景五：UI 设计与基础组件封装

目标：从灰蒙蒙的 Element Plus 默认样式，变成有品牌感的界面；前端基础组件一条指令生成。

第一步：设计系统生成

指令：

复制代码

```
Hify 是一个 AI Agent 开发平台，面向技术团队内部使用，用户是开发者。
```

```
管理后台为主——大量表格、表单、配置页，加一个对话页。
风格：浅底 + 科技感点缀。主色蓝紫系，辅色青色，
侧边栏深色底，按钮和关键元素用亮色。
参考 Linear、Supabase 的视觉风格——干净但不无聊。
帮我设计一套 CSS 变量设计系统：主色/背景色阶/文字色阶/圆角/阴影/过渡动效
```

要点：

- 展示指令的结构：产品定位 → 用户群体 → 风格方向 → 具体参考
- 拿到 CSS 变量后快速 review：颜色搭配、对比度、整体调性
- 演示把 CSS 变量写入 src/styles/variables.css 的过程

第二步：侧边栏改造演示

展示改造前后的对比，重点说明：

- 深色侧边栏 + 浅色内容区的层次感
- 选中态的左侧竖线设计
- Hify 品牌名的渐变文字效果

第三步：前端基础组件一条指令生成

指令：

复制代码

```
在 hify-web 中创建以下前端公共组件（src/components/）：
1. HifyTable.vue：通用列表表格，props 接收 columns 配置和 api 方法，
   内部管理 loading 和分页，暴露 refresh() 方法。
2. HifyFormDialog.vue：通用表单弹窗，v-model 控制显示，
   open(data?) 方法区分新增/编辑模式，提交触发 submit 事件。
3. useConfirm.ts：删除确认 composable，接收确认文案和 API 方法，
   一行代码完成确认→调接口→成功提示。
4. useRequest.ts：请求状态管理，返回 { data, loading, error, execute }。
5. notify.ts：统一通知封装，notifySuccess/notifyError/notifyWarning。
所有组件 Vue 3 Composition API + TypeScript，泛型支持不同数据类型。
```

要点：

演示生成后的组件代码，说明每个组件解决什么问题

重点展示 HifyTable 的 API 设计：传入 columns 配置和 API 方法，不需要在每个页面重复写分页逻辑

说明为什么一条指令生成五个组件：后端工程师不需要在前端组件细节上花时间，AI 做初稿，你验收好不好用

第四步：UI 打磨来回调的演示

用 ProviderList mock 页面演示三轮打磨：

第一轮：

```
ProviderList 的表格行高太高，改成 52px。
操作列的编辑和删除按钮间距太小，加 8px margin-left。
```

第二轮：

```
状态列的禁用标签用灰色，不要用红色。
禁用是正常状态，不是错误，不应该用 danger 色。
```

第三轮：

```
整体看一下 ProviderList：
1. 分页器居右对齐，上方加细分割线
2. 新增按钮加 Plus 图标
3. 操作列改成 text 类型按钮：编辑蓝色、删除红色
```

要点：

演示“哪里不对 + 应该怎样”的描述句式

强调不需要知道 CSS 属性名，只需要能说清楚你的判断

展示三轮之后页面的变化

总结

回顾这节实操课展示的五个场景，有几个核心动作值得你记住。

认知对齐先于执行。每个大任务开始前，先让 Claude Code 读一遍 CLAUDE.md，确认它理解了约束。这一步不做任何代码，只是对齐认知，但能帮你在执行阶段少走弯路。

报错信息要完整。启动失败、编译报错，把完整错误日志给它，而不只是最后一行报错。上下文越完整，它的修复准确率越高。两三轮还没解决，停下来自己看一眼，往往问题没你想的复杂。

SDD 闭环是日常。GlobalExceptionHandler 里硬编码错误码这个问题，发现了、改了、同步更新 CLAUDE.md，这就是 SDD 闭环在日常开发里的样子。不是什么宏大的流程，就是每次 Claude Code 跑偏时随手堵住同类问题的口子。

UI 打磨要敢说。看出来哪里不对，直接说：间距太大、颜色不对、弹窗太宽。
Claude Code 负责把你的判断翻译成代码，你不需要会写 CSS，但你需要有判断力。每一轮打磨都在训练这个判断力。

到这里，工程搭建篇完整结束。你手上现在有：前后端都能跑的工程骨架、三档就绪的后端基础组件、有品牌感的前端界面、一套可复用的前端公共组件。从下一讲开始，正式进入业务功能开发，第一讲是模型提供商管理。

期待你与我分享自己的体验。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (7)

最新　精选



北风一叶
2026-05-29 来自北京

这些讲读下来，给我的感觉 就是 ：
第一：你需要知道你即将做一个什么东西
第二：你要熟悉这其中的每一步
第三：把每一步细化成精准的提示词，让Ai能读懂的，比如代码要规范，就是一个很不好的提示词
第四： 要慢，稳步推进，慢就是快，Ai并非万能，它只是个工具

> 作者回复: 是的，这个是核心思路。就是先心里有数，再指挥AI。也就是驾驭AI。知道要做成什么样子，然后怎么做，然后让AI做

共 2 条评论>　　3



Luna
2026-06-21 来自广东

学这门CC课到现在，发现主要是经验和prompt，经验是要驾驭AI，做AI判官；prompt是要准确地对AI发号施令，让AI做好我想做的。

> 作者回复: 是的。我的体感下来也是经验+ prompt。
> 我的习惯就是想清楚，然后心里有底，知道让ai干什么，怎么干。然后AI执行，我review，然后让ai改。效率确实提高很多。做出来的东西质量比自己写的质量好，速度也快。

1



night
2026-05-28 来自上海

.claude 完成之后的几个动作，mvn ，基础框架，验证接口等，我一般直接让它自己生成task，或者说 前期是需求分析和设计，第二步我提供整体目标，让它做任务拆解，每个任务列清楚验证步骤和方法，我review task.md后，最爽的一步就是好现在按照 task.md 开始执行。现在在试加上自动 review 、子agent 以及skill化，目标实现较大程度的 yolo 😎

作者回复: 是的，这也是我最近写代码的主要姿势。
需求分析、设计、任务拆分都是它来。我只负责reveiw技术方案、逻辑实现步骤的准确性，如果没问题，就让它一把跑了。然后再review它生成的代码，做好单元测试和集成功能测试。代码和集成测试没问题后，就直接可以上线了。

这里需要注意：这一步做的好不好很依赖模型，模型能力强弱与否是这一步能否做的好的前提。

1



Geek_b2715e
2026-04-22 来自重庆

老师你用的什么模型生成的，速度这么快

作者回复: Claude Code 生成的



Poison
2026-04-22 来自北京

有视频讲解太棒了，老师后面会考虑出Claude的agent sdk企业级实战课程吗？期待可以再多一个项目哈哈

作者回复: 后面有机会可以试试，如果大家有需求的话，哈哈哈哈

Geek_347b1f
2026-04-16 来自乌克兰

老师能分享一下生成的代码吗

作者回复: 会有的，周末放出来～，放到github 上



Kyen
2026-04-15 来自江苏

太棒了，我觉得咱们课程最好的一点就是会有实操，手把手过一下整个流程！让我知道这些细节，不是只有理论。

作者回复: 嘿嘿，谢谢～～～



